

Modèle de Données (MVP) - SaaS Pharmacie

Ce document détaille le schéma relationnel optimisé pour le MVP de la plateforme. L'architecture est pensée pour répondre aux contraintes du marché africain : garantir l'isolation multi-tenant, faciliter la synchronisation "offline-first" et assurer une traçabilité sans faille (audit).

1. Principes Fondamentaux de l'Architecture

- Clés Primaires (UUID)** : Utilisation systématique d'UUID (v4) au lieu d'entiers auto-incrémentés. C'est indispensable pour générer des IDs en local sur la caisse (hors-ligne) sans risque de collision lors de la synchronisation avec le serveur.
- Isolation Multi-Tenant** : Présence obligatoire de la clé étrangère tenant_id sur absolument toutes les tables métiers (pour cloisonner les données de chaque pharmacie).
- Soft Deletes & Audit** : Présence des champs created_at, updated_at, et deleted_at (Soft Delete) partout. Les données ne sont jamais physiquement supprimées de la base pour des raisons d'audit et de conformité.

2. Dictionnaire des Données (Tables Clés)

Entité : Tenant (La Pharmacie)

Table globale gérant les abonnements des différentes pharmacies clientes.

Nom du Champ	Type (PostgreSQL)	Description & Contraintes
id	UUID	Clé primaire (PK)
name	VARCHAR	Nom de la pharmacie
status	VARCHAR	Ex: 'ACTIVE', 'SUSPENDED', 'TRIAL'

Entité : User (Employés & Dirigeants)

Comptes d'accès rattachés à une pharmacie spécifique.

Nom du Champ	Type (PostgreSQL)	Description & Contraintes
id	UUID	Clé primaire (PK)
tenant_id	UUID	Clé étrangère (FK) -> Tenant
role	VARCHAR	'ADMIN', 'PHARMACIST', 'CASHIER'
password_hash	VARCHAR	Mot de passe chiffré (bcrypt/argon2)

Entité : Product (Catalogue Médicaments)

Référentiel des produits. Le stock total est calculé dynamiquement via les Lots.

Nom du Champ	Type (PostgreSQL)	Description & Contraintes
id	UUID	Clé primaire (PK)
tenant_id	UUID	Clé étrangère (FK) -> Tenant
barcode	VARCHAR	Code barre unique par produit/tenant
name	VARCHAR	Nom (DCI ou nom commercial)
selling_price	DECIMAL(10,2)	Prix de vente public

Entité : Lot (Gestion des Péremptions & Stocks)

Crucial pour le suivi des dates d'expiration et la traçabilité des lots.

Nom du Champ	Type (PostgreSQL)	Description & Contraintes
id	UUID	Clé primaire (PK)
tenant_id	UUID	Clé étrangère (FK) -> Tenant
product_id	UUID	Clé étrangère (FK) -> Product
lot_number	VARCHAR	Numéro de lot fournisseur
expiration_date	DATE	Date de péremption
quantity	INTEGER	Quantité restante pour ce lot précis

Entités : Sale & SaleItem (Transactions Caisse)

Enregistrement des ventes. Si la caisse est hors-ligne, les IDs (UUID) sont générés sur la tablette et insérés ici lors du retour réseau.

Table	Champs Clés	Description
Sale	id, tenant_id, user_id, total_amount, status, payment_method	L'en-tête du ticket de caisse.
SaleItem	id, tenant_id, sale_id, lot_id, quantity, unit_price	Ligne de vente. Pointeur direct vers le lot_id pour déduire la quantité au bon endroit.

3. Exemple d'Implémentation NestJS / TypeORM (Multi-Tenant)

Pour s'assurer que les requêtes ne croisent jamais les données, chaque entité doit hériter d'une

classe de base abstraite.

```
import { PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn, DeleteDateColumn, Index } from 'typeorm';
```

```
export abstract class TenantBaseEntity {  
  @PrimaryGeneratedColumn('uuid')  
  id: string;
```

```
  @Index()  
  @Column({ type: 'uuid' })  
  tenant_id: string;
```

```
  @CreateDateColumn()  
  created_at: Date;
```

```
  @UpdateDateColumn()  
  updated_at: Date;
```

```
  @DeleteDateColumn()  
  deleted_at: Date;
```

```
}
```

```
// Exemple d'utilisation sur le Produit :  
import { Entity, Column } from 'typeorm';
```

```
@Entity('products')  
export class Product extends TenantBaseEntity {  
  @Column({ type: 'varchar', length: 255 })  
  name: string;
```

```
  @Column({ type: 'decimal', precision: 10, scale: 2 })  
  selling_price: number;  
}
```